Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Advanced Java (01CT1623)	**Aim:** Migrate the persistence layer of the Library Management System from JDBC (Task 1) to Hibernate ORM	
Take Home Exam 4	**Date :-22-04-2026**	**Enrollment No:-** 92301733024

HibernetUtil.java

```
package util;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

    private static final SessionFactory sessionFactory;

    static {
        try {
            sessionFactory = new Configuration()
                    .configure("hibernate.cfg.xml")
                    .buildSessionFactory();
        } catch (Exception e) {
            throw new ExceptionInInitializerError(e);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

BookDAO.java

```
package dao;

import model.Book;
import util.HibernateUtil;
import org.hibernate.*;

import java.util.List;

public class BookDAO {

    // SAVE
    public void saveBook(Book book) {
        Transaction tx = null;

        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            tx = session.beginTransaction();
```

```
        session.save(book);

        tx.commit();
    } catch (Exception e) {
        if (tx != null) tx.rollback();
        e.printStackTrace();
    }
}

// LIST
public List<Book> getAllBooks() {
    try (Session session = HibernateUtil.getSessionFactory().openSession()) {
        return session.createQuery("from Book", Book.class).list();
    }
}

// UPDATE
public void updateBook(Book book) {
    Transaction tx = null;

    try (Session session = HibernateUtil.getSessionFactory().openSession()) {
        tx = session.beginTransaction();

        session.update(book);

        tx.commit();
    } catch (Exception e) {
        if (tx != null) tx.rollback();
    }
}

// DELETE
public void deleteBook(int id) {
    Transaction tx = null;

    try (Session session = HibernateUtil.getSessionFactory().openSession()) {
        tx = session.beginTransaction();

        Book book = session.get(Book.class, id);
        if (book != null) {
            session.delete(book);
        }

        tx.commit();
    } catch (Exception e) {
        if (tx != null) tx.rollback();
```

```
        }
    }
}
```

ViewBooksServlet:

```
package servlet;

import dao.BookDAO;
import model.Book;
import model.BookBean;

import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;
import java.io.IOException;
import java.util.*;

public class ViewBooksServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session == null || session.getAttribute("user") == null) {
            response.sendRedirect("login.jsp");
            return;
        }

        BookDAO dao = new BookDAO();
        List<Book> books = dao.getAllBooks();
        request.setAttribute("bookList", books);

        List<BookBean> beanList = new ArrayList<>();

        for (Book b : books) {
            BookBean bean = new BookBean();
            bean.setBookId(b.getBookId());
            bean.setTitle(b.getTitle());
            bean.setAuthor(b.getAuthor());
            bean.setQuantity(b.getQuantity());
            beanList.add(bean);
        }
```

```
        request.setAttribute("bookList", beanList);

        RequestDispatcher rd = request.getRequestDispatcher("viewBooks.jsp");
        rd.forward(request, response);
    }
}
```

AddBookServlet:

```
package servlet;

import dao.BookDAO;
import model.Book;

import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;
import java.io.IOException;

public class AddBookServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session == null || session.getAttribute("user") == null) {
            response.sendRedirect("login.jsp");
            return;
        }

        String title = request.getParameter("title");
        String author = request.getParameter("author");
        int quantity = Integer.parseInt(request.getParameter("quantity"));

        BookDAO dao = new BookDAO();
        dao.saveBook(new Book(title, author, quantity));

        response.sendRedirect("viewBooks");
    }
}
```

MainApp

```
package main;
```

```
import dao.BookDAO;
import model.Book;

public class MainApp {
   public static void main(String[] args) {

      BookDAO dao = new BookDAO();

      dao.saveBook(new Book("Java Programming", "James Gosling", 10));

      dao.getAllBooks().forEach(b ->
            System.out.println(b.getBookId() + " " + b.getTitle())
      );

      Book book = dao.getAllBooks().get(0);
      book.setQuantity(20);
      dao.updateBook(book);

      dao.deleteBook(2);
   }
}
```

hibernate.cfg.xml:

```
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
   <session-factory>

      <!-- DB Connection -->
      <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
      <property name="hibernate.connection.url">jdbc:mysql://localhost:3307/librarydb</property>
      <property name="hibernate.connection.username">root</property>
      <property name="hibernate.connection.password"></property>

      <!-- Dialect -->
      <property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>

      <!-- Show SQL -->
      <property name="hibernate.show_sql">true</property>

      <!-- Auto create/update -->
      <property name="hibernate.hbm2ddl.auto">update</property>
```

```
    <!-- Entity Mapping -->
    <mapping class="model.Book"/>

  </session-factory>
</hibernate-configuration>
```

Comparison of Hibernate (ORM) and JDBC

1. Development Effort
JDBC requires a large amount of boilerplate code for establishing connections, executing queries, and handling result sets. Developers must manually map database records to Java objects.
In contrast, Hibernate significantly reduces development effort by automatically handling object-relational mapping. Developers work with Java objects instead of writing repetitive SQL code.

2. Query Language (SQL vs HQL)
JDBC uses SQL (Structured Query Language), which operates directly on database tables.
Hibernate uses HQL (Hibernate Query Language), which operates on Java objects (entities) instead of tables.
HQL is database-independent, while SQL is database-specific.

3. Caching
JDBC does not provide any built-in caching mechanism. Every query is executed directly on the database, which can reduce performance for repeated operations.
Hibernate provides built-in caching:

- First-level cache (Session level)
- Second-level cache (optional)

This improves performance by reducing unnecessary database queries.

4. Lazy Loading
JDBC does not support lazy loading. All data must be fetched immediately when a query is executed.
Hibernate supports lazy loading, which means data is fetched only when it is actually required. This optimizes performance and resource usage.

5. Maintainability
JDBC code is harder to maintain because SQL queries are scattered throughout the application and changes in database structure require multiple modifications.
Hibernate improves maintainability by using a clean object-oriented approach. Changes in the database can be handled with minimal code changes.

6. Suitability in J2EE Multi-Tier Applications
JDBC is suitable for small or simple applications where full control over SQL is required.
Hibernate is more suitable for enterprise (J2EE) multi-tier applications because it:

- Follows MVC architecture
- Reduces code complexity
- Improves scalability

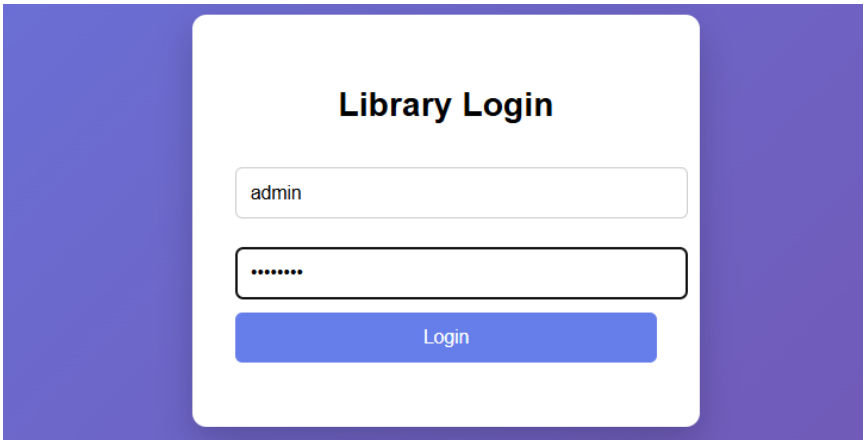
- Integrates well with frameworks

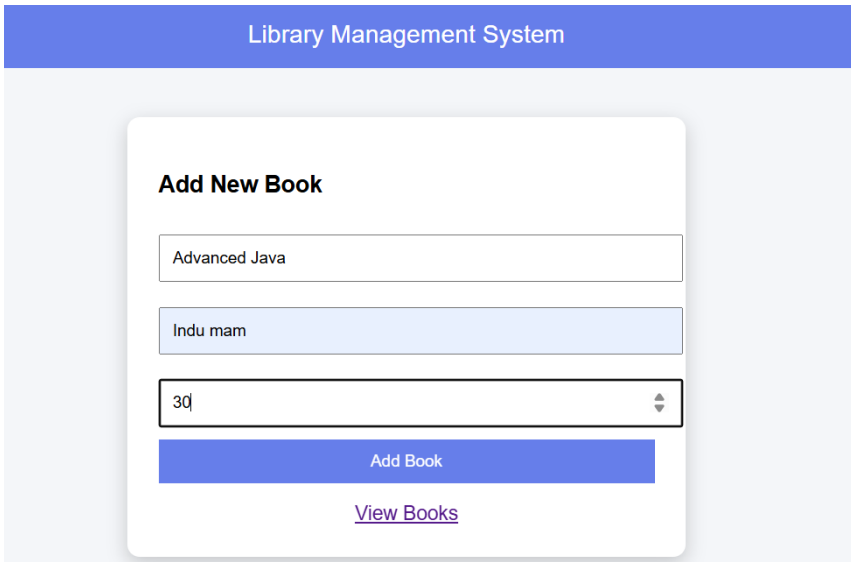
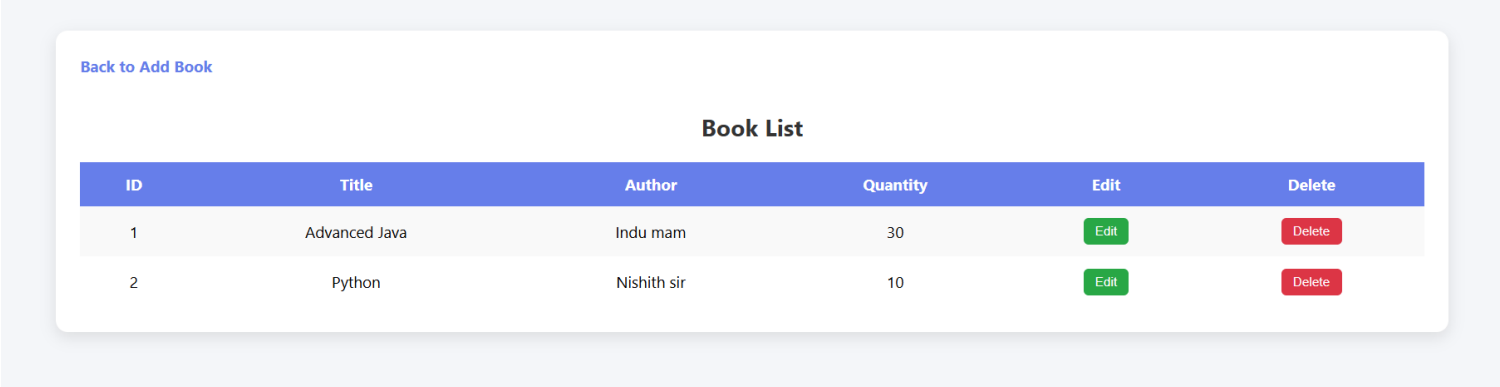
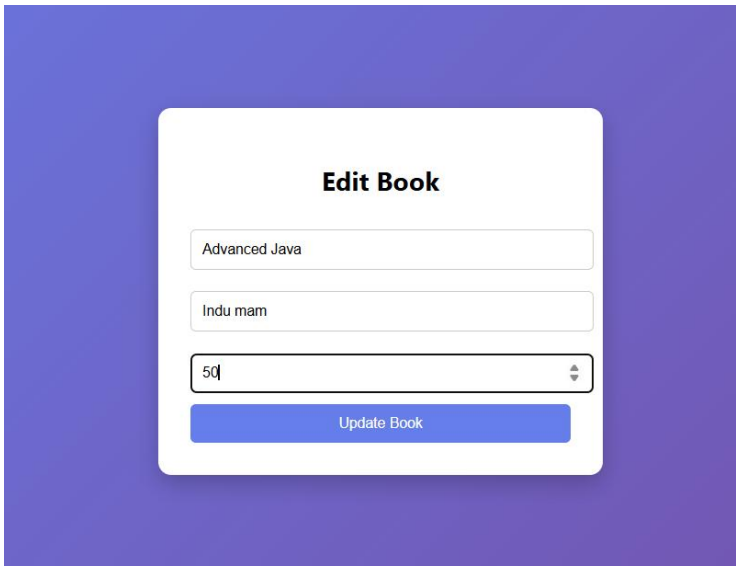
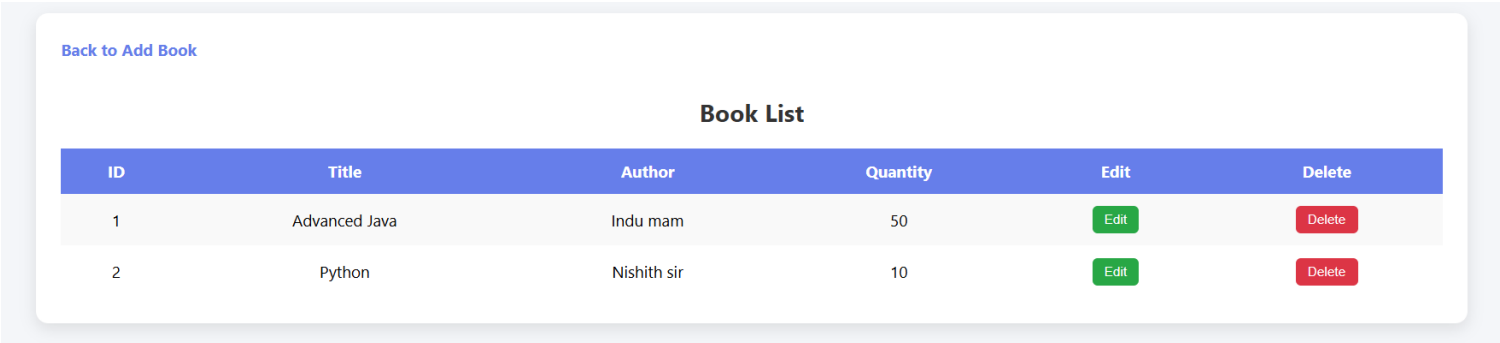
7. When to Prefer Hibernate

Hibernate should be preferred when:

- The application is large or complex
- Rapid development is required
- Maintainability is important
- Object-oriented design is needed
- Caching and performance optimization are required
- The application follows J2EE multi-tier architecture

OUTPUT:





Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Advanced Java (01CT1623)	**Aim:** Migrate the persistence layer of the Library Management System from JDBC (Task 1) to Hibernate ORM	
Take Home Exam 4	**Date :-22-04-2026**	**Enrollment No:-** 92301733024